

Repositorio Académico UPC

Módulos Federados: Implementación basada en el patrón de “Microfrontends” para potenciar la entrega continua y mantenimiento de una aplicación web

Item Type	info:eu-repo/semantics/bachelorThesis
Authors	Mamani Zuñiga, Carlos Alberto;Huaman Aparcana, Jose Luis
Publisher	Universidad Peruana de Ciencias Aplicadas (UPC)
Rights	info:eu-repo/semantics/openAccess
Download date	2026-09-02 02:43:53
Item License	http://creativecommons.org/licenses/by-nc-sa/4.0/
Link to Item	http://hdl.handle.net/10757/671607



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

PROGRAMA ACADÉMICO DE INGENIERÍA DE SOFTWARE

Módulos Federados: Implementación basada en el patrón de “Microfrontends”
para potenciar la entrega continua y mantenimiento de una aplicación web

TRABAJO DE SUFICIENCIA PROFESIONAL

Para optar el título profesional de Ingeniero de Software

AUTOR(ES)

Mamani Zuñiga, Carlos Alberto
Huaman Aparcana, Jose Luis

0009-0007-4744-5890
0009-0000-6237-847X

ASESOR(ES)

Subauste Oviden, Daniel Alejandro

0000-0003-1131-1384

Lima, 16 de noviembre de 2023

DEDICATORIA DE CARLOS ALBERTO MAMANI ZUÑIGA

Dedico este trabajo a mis padres, Hugo y Camila, por su inquebrantable apoyo y aliento a lo largo de este arduo viaje académico. A mi familia en general, por siempre haber estado ahí para alentarme y compartir alegrías y desafíos. A mi compañero/a de tesis, José, por compartir este desafío y ser colaborador excepcional en estos años de universidad. Agradezco sinceramente a los profesores de la carrera, por su guía experta y dedicación. Este logro es el resultado de nuestro esfuerzo conjunto

DEDICATORIA DE JOSÉ LUIS HUAMÁN APARCANA

Dedico este trabajo a toda mi familia por el apoyo constante en mi carrera académica, mis amigos por el apoyo moral, mis profesores, en especial a Wilder Namay, Walter Cueva, Luis Canaval, Angel Velazquez, Ricardo Gonzales y Jorge Mayta, por su tiempo invertido en su trabajo como docentes. Especiales agradecimientos a mi compañero Carlos Mamani por su dedicación en los trabajos y el ser una persona en la siempre pude confiar en mi trayecto universitario, y mi pareja Jimena Ruiz por ser un pilar de soporte y confianza en estos últimos años.

AGRADECIMIENTOS

Queridos profesores y querida Universidad Peruana de Ciencias Aplicadas,

Con profundo agradecimiento y gratitud, dedicamos esta tesis de Ingeniería de Software a todos ustedes, quienes han sido pilares fundamentales en nuestra formación académica. A lo largo de este arduo pero enriquecedor camino, su dedicación, conocimiento y apoyo incondicional han sido la fuerza impulsora que nos han guiado hacia el logro de este importante hito.

Agradecemos al profesor Willy Ugarte y Luis Canaval por su compromiso con la excelencia académica, guía experta y apertura a nuevas ideas.

Asimismo, extendemos el reconocimiento a la Universidad Peruana de Ciencias Aplicadas por brindarnos la oportunidad de crecer académica y personalmente en un entorno que fomenta la innovación, la investigación y la excelencia.

Este trabajo de suficiencia académica es el resultado de la combinación de sus enseñanzas, consejos e inspiración. Agradecemos sinceramente el impacto positivo que han tenido en nuestro desarrollo como profesionales y les dedicamos este logro con la esperanza de que nuestra contribución al campo de la Ingeniería de Software esté a la altura de la calidad educativa que hemos recibido.

Con profundo agradecimiento,

José Huamán Aparcana y Carlos Mamani Zúñiga

RESUMEN

En el presente trabajo de suficiencia profesional se presenta un modelo de arquitectura basada en el patrón de “Microfrontends” con módulos federados, el cual fue implementado para una plataforma de gestión financiera perteneciente a una institución bancaria en respuesta a la necesidad de mejorar el mantenimiento y desarrollo de nuevas características.

Para este proyecto partimos de una plataforma ya implementada bajo una arquitectura de “Microfrontends” y conformada por varios componentes. Esto permitió que cada uno exista de forma independiente del resto en cuanto a su ciclo de vida como software, y a nivel de negocio, se agrupen en los múltiples servicios que ofrece la institución al representar una única funcionalidad. Como parte de la mejora propuesta, se incorporó el uso de módulos federados con el objetivo de cargar de forma dinámica dependencias desde el componente padre, lo que resultó en la estandarización de bibliotecas orientadas a mejorar los tiempos y calidad del desarrollo de nuevas características. Esto requirió mantener un enfoque en el flujo de desarrollo con el uso de herramientas de registro de dependencias, revisión de código y automatización del despliegue. Finalmente, se implementaron una serie de bibliotecas de componentes, así como scripts utilitarios que sirvan para la creación y mantenimiento de nuevos “Microfrontends”.

Palabras clave: módulos federados; Microfrontends; arquitectura escalable; estandarización de dependencias.

Federated Modules: Implementation based on the Microfrontends Pattern to Enhance Continuous Delivery and Maintenance of a Web Application

ABSTRACT

In this professional proficiency work, we present an architecture model based on the Microfrontends pattern with federated modules, which was implemented for a financial management platform belonging to a banking institution in response to the need to improve maintenance and develop new features. For this project, we started with a platform already implemented under a Microfrontends architecture and composed of various components. This allowed each component to exist independently from the rest in terms of its software lifecycle, and at the business level, they were grouped into the multiple services offered by the institution, representing a single functionality. As part of the proposed improvement, federated modules were incorporated to dynamically load dependencies from the parent component, resulting in the standardization of libraries aimed at improving the development of new features in terms of speed and quality. This required maintaining focus on the development cycle with the use of tools such as a package register, code review and deploy automatizations. Finally, components libraries were implemented as well as scripts used to create and maintain new Microfrontends.

Keywords: federated modules; micro-frontends; scalable architecture; dependencies standardization.

U201711716_JOSE LUIS HUAMAN APARCANA_Módulos Federados: Implementación basada en el patrón de "Microfrontends" para potenciar la entrega continua y mantenimiento de una aplicación web

INFORME DE ORIGINALIDAD

2%	2%	1%	2%
INDICE DE SIMILITUD	FUENTES DE INTERNET	PUBLICACIONES	TRABAJOS DEL ESTUDIANTE

FUENTES PRIMARIAS

1	Submitted to Universidad Peruana de Ciencias Aplicadas Trabajo del estudiante	1%
2	repositorio.urp.edu.pe Fuente de Internet	<1%
3	Submitted to Centro Europeo de Postgrado - CEUPE Trabajo del estudiante	<1%
4	Ignacio Lacalle Úbeda. "Diseño y desarrollo de una arquitectura de Internet de las Cosas de Nueva Generación orientada al cálculo y predicción de índices compuestos aplicada en entornos reales", Universitat Politècnica de Valencia, 2022 Publicación	<1%
5	Casimiro Canales, Tania Mirella. "Eliminando Fricciones en Inclusión Financiera:	<1%

TABLA DE CONTENIDOS

<u>1</u>	<u>CAPÍTULO 1: DEFINICIÓN DEL PROYECTO</u>	11
1.1	<u>ANTECEDENTES</u>	11
1.2	<u>DESCRIPCIÓN DE LA ORGANIZACIÓN</u>	11
1.3	<u>ANÁLISIS DEL PROBLEMA</u>	12
1.4	<u>OBJETIVOS</u>	13
1.4.1	<u>General</u>	13
1.4.2	<u>Específicos</u>	13
1.5	<u>INDICADORES DE ÉXITO</u>	14
1.6	<u>PLANIFICACIÓN DEL PROYECTO</u>	14
1.6.1	<u>Equipo</u>	14
1.6.2	<u>Tiempos</u>	15
1.6.3	<u>Metodología</u>	17
1.6.4	<u>Tecnologías</u>	17
<u>2</u>	<u>CAPÍTULO 2: MARCO TEÓRICO</u>	17
2.1	<u>RELACIONADO A LA INDUSTRIA Y PROCESOS DE NEGOCIO</u>	17
2.2	<u>RELACIONADO A LA INGENIERÍA DE SISTEMAS</u>	18
2.3	<u>CUMPLIMIENTO NORMATIVO</u>	20
<u>3</u>	<u>CAPÍTULO 3: DESARROLLO DEL PROYECTO</u>	21
3.1	<u>DISEÑO DE LA SOLUCIÓN</u>	21
3.1.1	<u>Arquitectura de Negocio</u>	21
3.1.2	<u>Arquitectura de Datos</u>	23
3.1.3	<u>Arquitectura de Aplicaciones</u>	25
3.1.4	<u>Arquitectura Tecnológica</u>	25
3.1.5	<u>Arquitectura Integrada</u>	27
3.2	<u>DESARROLLO DE LA SOLUCIÓN</u>	28
3.3	<u>GESTIÓN DEL PROYECTO</u>	32
3.3.1	<u>Alcance del proyecto</u>	32
3.3.2	<u>Hoja de ruta del proyecto</u>	33
3.3.3	<u>Lista de Hitos</u>	33
3.3.4	<u>Matriz de asignación de responsabilidades</u>	33

3.3.5	Gestión de calidad	33
3.3.6	Gestión de adquisiciones	34
3.3.7	Gestión de los recursos	34
3.3.8	Gestión de los riesgos	35
3.3.9	Gestión de requerimientos	35
3.3.10	Gestión de alcance	35
3.3.11	Gestión de costo	36
3.3.12	Gestión del cronograma	36
3.3.13	Gestión del proyecto	36
3.4	RESULTADO DEL PROYECTO	36
4	CONCLUSIONES Y RECOMENDACIONES	37
4.1	CONCLUSIONES	37
4.2	RECOMENDACIONES	38
5	REFERENCIAS	39
6	ANEXOS	41
6.1	PLAN DE CONTINUIDAD	41
6.2	PLAN DE COSTOS	42

ÍNDICE DE FIGURAS

Figura 1	12
Figura 2	14
Figura 3	21
Figura 4	22
Figura 5	23
Figura 6	24
Figura 7	25
Figura 8	26
Figura 9	28
Figura 10	30
Figura 11	31
Figura 12	31

ÍNDICE DE TABLAS

<u>Tabla 1</u>	15
<u>Tabla 2</u>	41
<u>Tabla 3</u>	42

CAPÍTULO 1: DEFINICIÓN DEL PROYECTO

Antecedentes

Una determinada institución bancaria, se vale del uso de una plataforma de gestión financiera, para brindar y administrar servicios financieros respectivamente. Inicialmente estas fueron planteadas como proyectos de software monolíticos por su rapidez de desarrollo, sin embargo, conforme se agregaban funcionalidades y aumentaba su demanda de uso, se volvieron enormemente complejos y difíciles de mantener. Cada vez que se realizaba un cambio en una parte de la aplicación se corría el riesgo de afectar otras áreas de manera inesperada, lo que resultaba en un ciclo constante de pruebas y correcciones. Esto aumentaba el tiempo, los costos de desarrollo y mantenimiento de manera significativa.

En respuesta a estos problemas, la institución decidió adoptar una arquitectura basada en “Microfrontends”. En primer lugar, este tipo de arquitectura permitiría dividir una aplicación en módulos más pequeños e independientes, lo que facilitaría el desarrollo y mantenimiento al permitir que equipos especializados se encargaran de cada módulo o “Microfrontend” de manera aislada. Esto reduciría el riesgo de impactos colaterales al realizar cambios en la aplicación. Además, la implementación y el despliegue serían más ágiles, ya que los módulos podrían actualizarse de forma independiente, sin afectar a otras partes de la aplicación. Esta modularidad también mejoraría la escalabilidad, ya que se podrían agregar o eliminar componentes según las necesidades de la empresa, lo que permitiría un crecimiento más eficiente.

Actualmente, se apunta a mejorar el control de las dependencias que usa cada módulo y crear una serie de bibliotecas estándares que reduzcan los tiempos de entrega y mantengan una misma calidad de código en cada componente.

Descripción de la Organización

Es una institución bancaria líder que se destaca por su compromiso con la excelencia en la prestación de servicios y soluciones financieras integrales. Su misión es proporcionar soluciones financieras flexibles y confiables que satisfagan las necesidades individuales y comerciales de sus clientes, respaldando sus objetivos financieros a corto y largo plazo.

Algunos de los servicios de ofrecer la instrucción son los siguientes:

- Préstamos y créditos, que brindan la posibilidad de adquirir bienes raíces, automóviles u otros activos importantes respaldados por plazos y tasas competitivas.

- Tarjetas de crédito y débito para transacciones del momento y compras en línea. Y
- Productos de inversión, como certificado de depósito e inversión en la bolsa, permitiendo a sus clientes maximizar sus ganancias utilizando estrategias de crecimiento financiero.

Análisis del Problema

Con el objetivo de analizar el problema identificado, se utilizará el diagrama de Ishikawa descrito en la figura 1 para identificar y visualizar las principales causas que han contribuido con la desorganización en la implementación de nuevas características en la plataforma.

En primer lugar, la gestión de proyectos inadecuada juega un papel crucial en esta situación. La planificación a corto plazo, evaluación de riesgos y los cambios frecuentes en el alcance del proyecto pueden llevar a una ejecución caótica en la que los distintos equipos ejecuten una solución desincronizada y apresurada.

La cultura organizacional también desempeña un papel significativo. La falta de una cultura de colaboración y aprendizaje continuo puede impedir la comunicación efectiva entre los equipos, mientras que la resistencia al cambio puede generar una evolución lenta de la solución. La rotación frecuente de personal complica estos desafíos, ya que la pérdida de conocimientos y experiencia obstaculiza aún más la implementación de nuevos estándares y tecnologías.

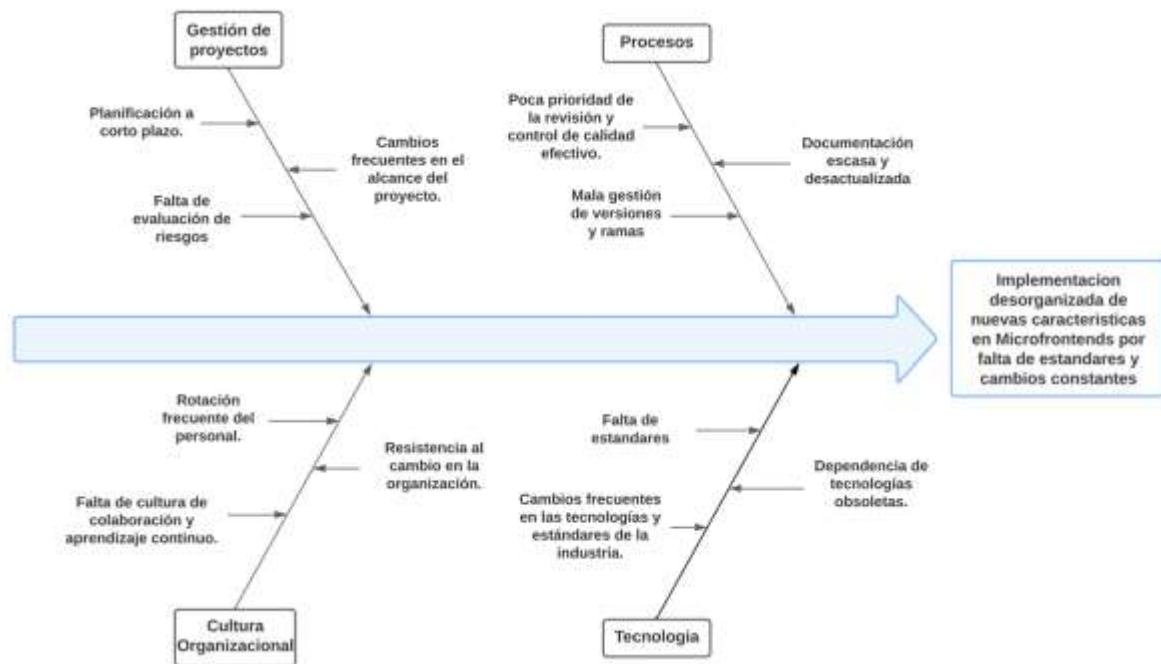
En cuanto a los procesos, la poca prioridad de la revisión y control de calidad puede dar lugar a problemas no detectados en las nuevas implementaciones. La documentación escasa y desactualizada dificulta la comprensión y el mantenimiento del código, lo que, junto al mal control de versiones ocasiona demoras en la implementación de nuevas características.

En el ámbito tecnológico, la ausencia de estándares puede aumentar el riesgo de errores humanos y una baja calidad. La dependencia de tecnologías obsoletas o la adopción constante de nuevas tecnologías sin el conocimiento adecuado de estas nuevas tecnologías pueden llevar a problemas de compatibilidad y mantenimiento.

Abordar estos problemas en los procesos y la tecnología es esencial para lograr una implementación exitosa y organizada de nuevas características.

Figura 1

Diagrama de Ishikawa de las causas identificadas sobre el problema



Objetivos

General

Implementar una arquitectura de “Microfrontends” utilizando módulos federados que permita la estandarización de bibliotecas y así mejorar los tiempos y calidad de desarrollo de nuevas características de una plataforma de gestión financiera en el contexto de una institución bancaria.

Específicos

OE1: Analizar la implementación actual de la plataforma y sus componentes junto con sus principales problemas, para identificar puntos de mejora. Estos últimos se categorizarán por área de uso para posteriormente implementar mejoras de forma gradual.

OE2: Diseñar una arquitectura, basada en proyectos “Microfrontends” ya existentes y los puntos de mejora identificados, con uso de módulos federados.

OE3: Implementar la arquitectura diseñada de forma gradual en cada uno de los componentes por área identificadas.

OE4: Validar la implementación de la arquitectura de forma gradual en base a métricas asociadas a los problemas identificados que nos ayuden a cuantificar el impacto de las mejoras.

Indicadores de Éxito

IE1: Aprobación del área de gerencia y el mánager de TI del análisis de la problemática y mejoras sobre las aplicaciones web implementadas actualmente.

IE2: Aprobación del arquitecto “Frontend” del diseño de la solución en base a la creación de diagramas de arquitectura aplicaciones, negocio, tecnología, y demás artefactos necesarios.

IE3: Finalización de la implementación de la solución por parte del área de desarrollo.

IE4: Aprobación de las áreas de la entidad bancaria en donde se utilizan las aplicaciones en las cuales se implementó la solución de mejora

Planificación del Proyecto

Equipo

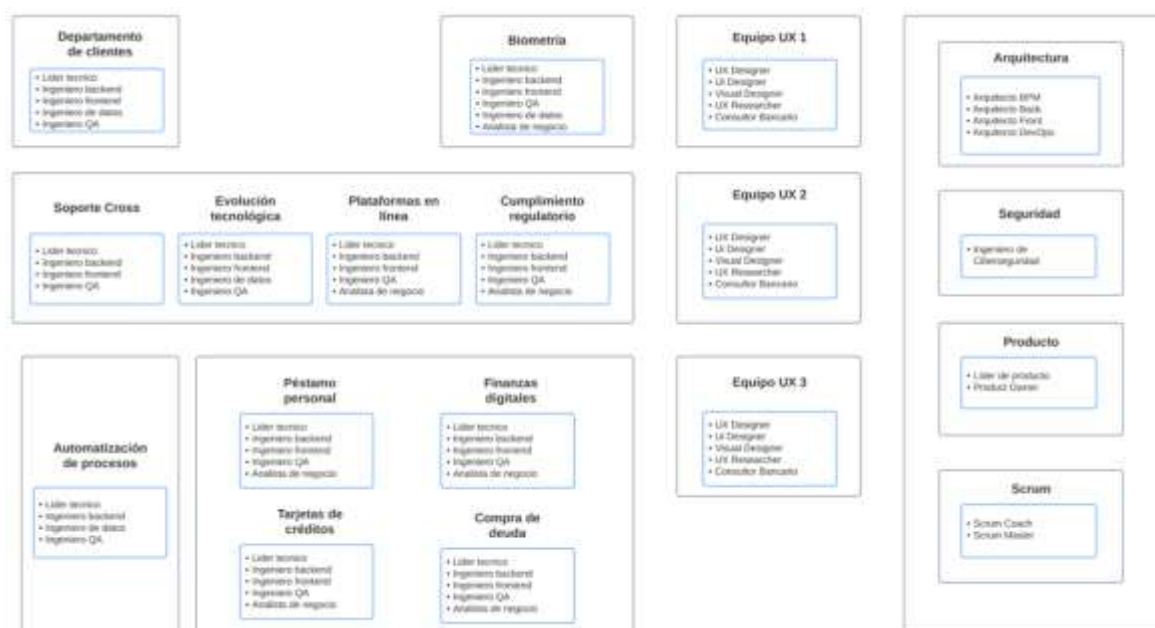
La conformación del equipo está dividida por productos financieros, diagramada en la figura 2. Para garantizar un entendimiento completo de reglas y conceptos financieros cada equipo tiene disponible un Analista del negocio, esta persona asesora al equipo con su conocimiento para resolver consultas de manera eficiente.

Por otro lado, algunos equipos cuentan con Ingenieros de datos, quienes realizan la limpieza y transformación e integración de datos dentro de la organización. Otro factor importante es la experiencia de usuario, para ello el equipo UX trabaja de manera cruzada en múltiples proyectos realizando el diseño de las interfaces para lograr una experiencia correcta para los usuarios.

En un nivel más completo, se tiene el equipo de scrum para la garantizar que el trabajo este alineado en los principios ágiles. El equipo de producto, que rige como el responsable para generar el máximo valor en el producto. El equipo de seguridad, encargado de realizar auditorías en el código para detectar vulnerabilidades. Y el equipo de arquitectura, encargado de la definición y diseño de toda la infraestructura, asegurando que sea eficiente, seguro y escalable

Figura 2

Diagrama de las áreas de desarrollo divididas por funcionalidad



Tiempos

Con respecto a la gestión de tiempos, según se muestra en la tabla 1, la institución divide el año en 4 trimestres, definiendo los principales incrementos que se realizará en los productos planificando las tareas con un trimestre de anticipación. A su vez, estos trimestres se dividen en 6 sprints de 2 semanas de duración cada uno, donde se detalla una priorización de las tareas en función a la capacidad del equipo en ese momento y las restricciones que pueden generar las tareas.

Una vez que termina el periodo de desarrollo de una funcionalidad inicia el periodo de pruebas, este proceso inicia en el siguiente sprint, permitiendo al equipo de desarrollo continuar con el desarrollo de nuevos módulos, brindando el soporte a pruebas simultáneamente.

Tabla 1

Tabla de la organización de sprints por año laboral

Indice	Nombre de la tarea	Fecha de inicio	Fecha final	Duración (días)
1	Implementación basada en el patrón de “Microfrontends” para potenciar la entrega continua y mantenimiento de una aplicación web	03/01/2022	16/12/2022	347
1.1	Planificación	03/01/2022	04/02/2022	32

1.1.0	Análisis de la implementación actual de la plataforma	03/01/2022	07/01/2022	4
1.1.1	Validación de gerencia de proyectos	07/01/2022	07/01/2022	0
1.1.2	Product Backlog	10/01/2022	04/02/2022	25
1.2	Gestión de proyecto	07/03/2022	01/04/2022	25
1.2.0	Antecedentes	07/03/2022	11/03/2022	4
1.2.1	Marco Teórico	14/03/2022	18/03/2022	4
1.2.2	Alcance del proyecto	14/03/2022	18/03/2022	4
1.2.3	Hoja de ruta del proyecto	14/03/2022	18/03/2022	4
1.2.4	Lista de hitos	14/03/2022	18/03/2022	4
1.2.5	Matriz de asignación de responsabilidades	21/03/2022	25/03/2022	4
1.2.6	Gestión de calidad	21/03/2022	25/03/2022	4
1.2.7	Gestión de adquisiciones	21/03/2022	25/03/2022	4
1.2.8	Gestión de los recursos	21/03/2022	25/03/2022	4
1.2.9	Gestión de los riesgos	28/03/2022	01/04/2022	4
1.3.0	Gestión de los requerimientos	28/03/2022	01/04/2022	4
1.3.1	Gestión de alcance	28/03/2022	01/04/2022	4
1.3.2	Gestión de costo	28/03/2022	01/04/2022	4
1.3.3	Gestión de cronograma	28/03/2022	01/04/2022	4
1.3.4	Gestión del proyecto	28/03/2022	01/04/2022	4
1.3	Desarrollo	04/04/2022	16/12/2022	256
1.3.0	Desarrollo del 16hell16to Q1	04/04/2022	01/07/2022	88
1.3.1	Desarrollo del 16hell16to Q2	04/07/2022	30/09/2022	88
1.3.2	Desarrollo del 16hell16to Q3	03/10/2022	16/12/2022	74
1.4	Despliegue	28/11/2022	16/12/2022	18

1.4.0	Entrega documentación de microfrontends	28/11/2022	09/12/2022	11
1.4.1	Despliegue de microfrontends	12/12/2022	16/12/2022	4

Metodología

El flujo de trabajo está basado en la filosofía Ágil, la cual, según su manifiesto Beck et al. (2001), estipulan el desarrollo de soluciones de software de forma incremental e iterativa. Dentro de este, se encuentran marcos de trabajo como SCRUM y Kanban que precisan que pasos seguir para entregar un producto al cliente de forma eficiente. Por un lado, SCRUM provee una estructura de trabajo en una serie de sprints (iteraciones) en donde se involucran miembros como el Scrum Team, Scrum Master y Product Owner para implementar un proyecto plasmado en un Product Backlog (Schwaber & Sutherland, 2020). En cuanto a Kanban, este recomienda la visualización de los flujos de trabajo por medio de un tablero en donde se definan las tareas, los estados de cada y en qué punto se consideran concluidas.

Tecnologías

- Aplicaciones cliente: Angular 15, Webpack y Typescript
- Pruebas: Jest.js y SonarCube
- Computación en la nube: AWS
- Virtualización: Docker y Kubernetes
- Herramientas de gestión: Atlassian Suit
- Gestion de versiones: Git y Bitbucket

CAPÍTULO 2: MARCO TEÓRICO

Relacionado a la industria y procesos de negocio

Institución bancaria

Según glosario del Banco Central de Reserva del Perú (Banco Central de Reserva del Perú, 2022), una institución bancaria es una entidad cuyas operaciones y servicios pertenecen al rubro financiero. Esta genera dinero por medio de depósitos del público y otras fuentes, y lo utiliza, junto con su capital propio, para ofrecer mecanismos financieros y realizar inversiones. Adicionalmente, genera ganancias por medio de sus operaciones crediticias.

Producto Financiero

Son servicios enfocados en cubrir la demanda financiera del público y que representan los principales servicios que ofrecen las instituciones financieras. Es decir, es todo servicio de financiación ofrecido por las entidades financieras, con el objetivo de proporcionar capital o fondos a sus clientes para sus actividades o necesidades financieras (Asociación de Jóvenes Empresarios, 2009).

- **Préstamo:** Producto financiero que permiten a los individuos o empresas obtener una suma de dinero específica de una entidad financiera o prestamista a cambio de comprometerse a devolver ese monto junto con intereses en un período de tiempo acordado.
- **Tarjeta de Crédito:** Herramienta financiera que permite a los titulares realizar compras o retiros de efectivo utilizando un crédito otorgado por una entidad financiera o banco emisor. Los titulares de tarjetas pueden gastar hasta un límite de crédito predeterminado, y luego deben reembolsar el saldo acumulado en su totalidad o en pagos mensuales mínimos, generalmente con intereses.
- **Compra de Deuda:** La compra de deuda implica que una entidad adquiere deudas pendientes de individuos o empresas a menudo por un monto menor al valor original. Esto permite a los deudores transferir su deuda a nuevos acreedores, quienes pueden buscar recuperarla o negociar términos de pago más favorables. La compra de deuda es una práctica común en la industria financiera y de cobro.

Finanzas Digitales

Las finanzas digitales normalmente se refieren a la oferta de aplicaciones financieras a través de internet por parte de una organización y ofrecen un mayor alcance que los procesos tradicionales. Estos servicios están enfocados en un uso público general por medio de funcionalidades personalizables y amigables para el usuario (Hu & Zheng, 2016). Ejemplos de estas funciones incluyen banca en línea, pagos electrónicos e inversiones digitales

Relacionado a la Ingeniería de Sistemas

Frontend

En un aspecto general, se refiere a la interfaz de un software diseñada para ser el punto de interacción de un usuario. Para el caso de este trabajo, se definirá el término como una aplicación web en donde un cliente (browser) recibe archivos HTML, CSS y JavaScript, los cuales permiten la interacción por medio de código que ejecuta el cliente (Peltonen et al., 2021).

- Angular: Es una plataforma de desarrollo aplicaciones web basado en Typescript y orientada a crear Single Page Applications (SPA) eficientes. Hoy en día Angular se encuentra en su versión 16.2.3, sin embargo, para este proyecto se utilizará la 15.2.0. Sus principales características, según las definen en su documentación (Google, 2023), son: uso de componentes, colección de bibliotecas (navegación, manejo de formularios, comunicación cliente/servidor, etc) y un entorno rico de herramientas de desarrollo.
- Microfrontends: Es una técnica basada en descomponer aplicaciones frontend monolíticas en pedazos más simples que puedan ser desarrolladas, probadas y desplegadas independientemente, mientras que los usuarios mantengan un único producto cohesivo. Esta fue definida por Martin Fowler (Jackson, 2019) como: “Un estilo arquitectónico donde componentes independientes son compuestas en una aplicación”. Los principales beneficios de esta arquitectura son: actualizaciones incrementales, bases de código simples desacopladas, y despliegues independientes.
- Módulos Federados: Es una arquitectura, inicialmente desarrollada por Zack Jackson como una característica de webpack, la cual permite a una aplicación cargar código de otra y de esa forma consumir sus dependencias o componentes (Jackson, 2023). Esta técnica es utilizada en conjunto con micro-frontends, integraciones de múltiples aplicaciones o bibliotecas, para aprovechar sus beneficios.

Computación en la nube

La documentación de Amazon Web Services (Amazon Web Services, 2023), el proveedor de nube elegido en este proyecto define a la computación en la nube como el consumo en demanda de recursos de TI a través de internet, normalmente bajo un modelo de pago por unidad o tiempo de procesamiento. Existen 3 principales subcategorías de este tipo de servicios:

- Software as a Service (SaaS): Software o aplicaciones dedicadas
- Infrastructure as a Service (IaaS): Servicio de infraestructura de bajo nivel en la nube
- Platform as a Service (PaaS): Plataforma de servicios computacionales en la nube administrados por funcionalidades

En específico, este proyecto utilizara la nube bajo los modelos IaaS y PaaS. Lo mayores beneficios de utilizar este tipo de recursos, como los describen en su documentación, son los gastos variables en función del consumo, beneficios económicos gracias a la escala de su nube, mayor velocidad y agilidad, cero gastos en mantenimiento de centros de datos, escalabilidad global y recursos a demanda.

Cumplimiento normativo

La Ley N° 2670 (Ley General Del Sistema Financiero y Del Sistema de Seguros y Orgánica de La Superintendencia de Banca y Seguros, 1996), decretada por el congreso y presidente del Perú, establece las normas generales y específicas que rigen a las empresas del sistema financiero y de seguros, así como a la SBS. (Congreso de la República de Perú, 1996, Ley 2670).

- Gestión de Riesgos Legales y Regulatorios: Proceso de identificación, evaluación y mitigación de riesgos legales y regulatorios que podrían afectar las operaciones bancarias, asegurando el cumplimiento con leyes y regulaciones aplicables.
- Diseño de Políticas y Procedimientos: Creación y establecimiento de directrices y procesos internos que guían el comportamiento y las operaciones de la entidad bancaria para asegurar la consistencia y el cumplimiento normativo.
- Monitoreo y Control: Supervisión continua de las actividades bancarias y aplicaciones de medidas de control para detectar desviaciones de normativas, minimizar riesgos y garantizar el cumplimiento con estándares establecidos.
- Capacitación y Concientización: Proceso de educación y formación del personal en las políticas, regulaciones y mejores prácticas para crear conciencia sobre el cumplimiento normativo y la importancia de la seguridad.
- Prevención de Lavado de Dinero (PLD): Conjunto de medidas y procedimientos implementados para prevenir y detectar intentos de utilizar el sistema financiero para legitimar ganancias ilícitas, garantizando transparencia en las transacciones.
- Gestión de Riesgos Financieros: Evaluación y control de riesgos relacionados con la exposición a fluctuaciones financieras, incluyendo riesgos crediticios, de mercado y de liquidez, para salvaguardar la estabilidad financiera del banco.
- Protección de Datos y Privacidad: Implementación de medidas para resguardar la información personal y financiera de los clientes, garantizando su confidencialidad y cumpliendo con regulaciones de protección de datos.
- Tecnología y Cumplimiento Normativo: Integración de soluciones tecnológicas para automatizar, monitorear y asegurar el cumplimiento normativo, incluyendo procesos de auditoría, análisis de datos y gestión de riesgos.

CAPÍTULO 3: DESARROLLO DEL PROYECTO

Diseño de la Solución

Esta sección muestra el diseño de una arquitectura de “Microfrontends” utilizando módulos federados, se realizó la arquitectura de negocio en donde se describen la arquitectura de negocio, datos, entidades, tecnologías e integrada.

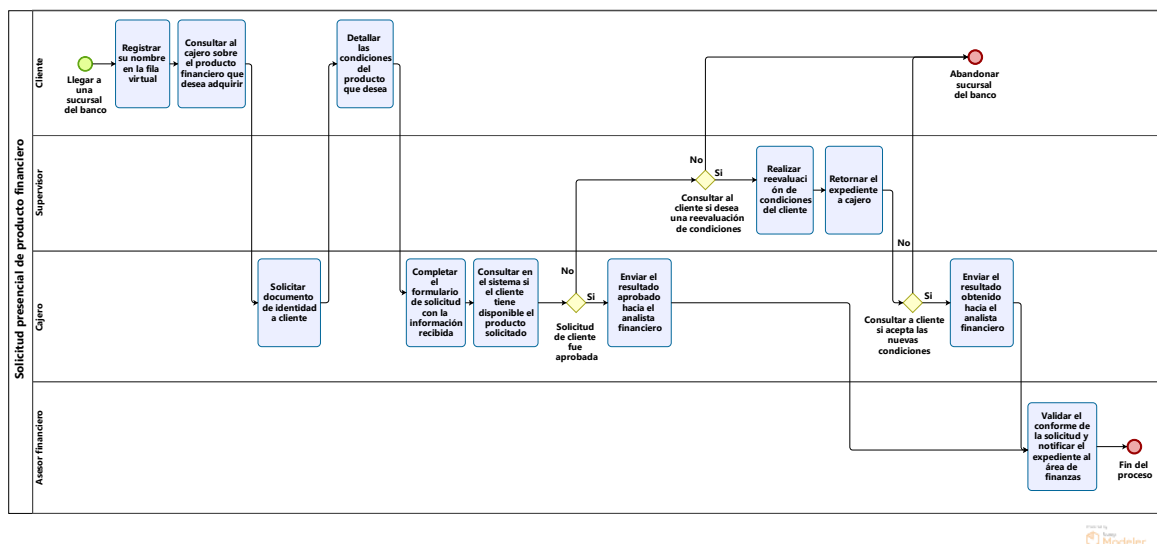
Arquitectura de Negocio

El modelo de Business Process Model Notation (BPMN) provee un gráfico que permite especificar procesos de negocios en un Business Process Diagram con el objetivo de comprender definir una notación estándar comprensible tanto para negocio como para la parte técnica (Object Management Group, 2023). Se detallarán los procesos de solicitud presencial, virtual y promoción de un producto financiero.

La figura 3 muestra el modelo BPMN del proceso de solicitud presencial de un producto financiero. Este inicia cuando el cliente se registra en la fila virtual de una agencia. Una vez es llamado a ventanilla, se selecciona el producto financiero que desea adquirir y un responsable completa un formulario de solicitud dentro de la plataforma para enviarlo al área de resolución. Allí un analista de riesgos valida las condiciones solicitadas y, si son aceptadas, se da la conformidad y se envía el expediente hacia el área de finanzas, terminando el flujo. En el caso de que las condiciones sean rechazadas, el analista retorna la solicitud al responsable de ventanilla con las condiciones que permitan la aprobación del producto solicitado para que sean comunicadas al cliente. En el caso que el cliente acepte la adquisición del producto con las nuevas condiciones, se continua con el flujo establecido o se termina con la atención del cliente.

Figura 3

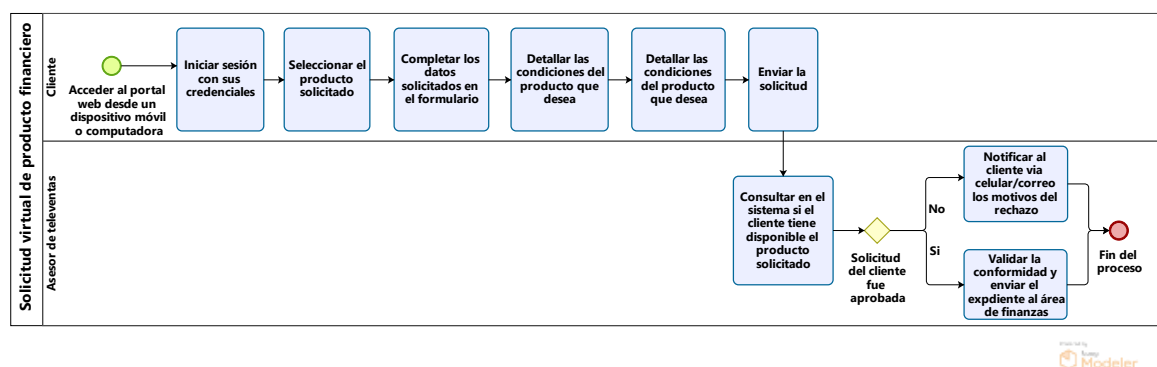
Diagrama BPM para flujo solicitud presencial de producto financiero



La figura 4 muestra el proceso de la solicitud virtual de un producto financiero. Para esto el cliente inicia accediendo a la plataforma desde su dispositivo móvil o computadora a la web. A continuación, selecciona el producto financiero que desea adquirir, completa los detalles necesarios en un formulario y lo envía. Posteriormente un asesor de televentas recibe la solicitud del producto, y realiza una validación de las condiciones. De ser validado, se envía un correo al cliente indicando la aprobación, y en caso contrario, los motivos del rechazo.

Figura 4

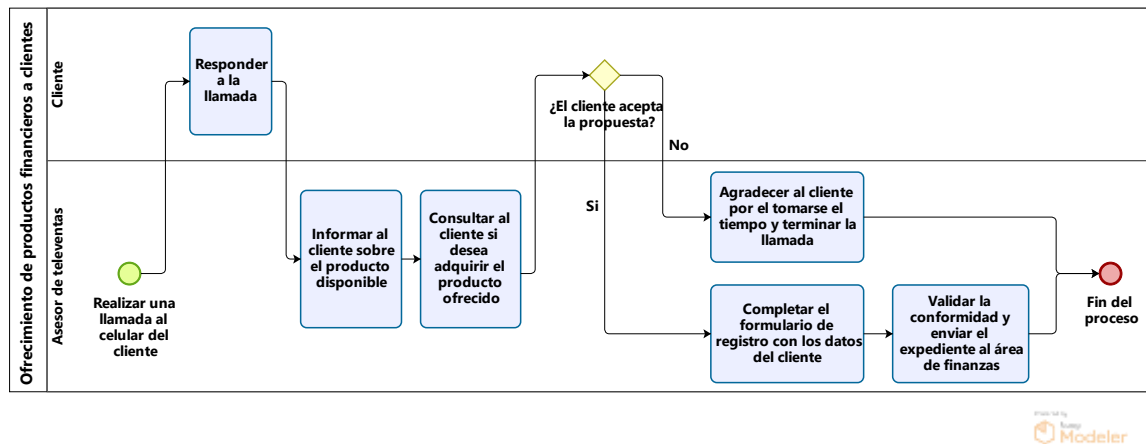
Diagrama BPM para flujo solicitud virtual de producto financiero



La figura 5 muestra el proceso de promoción de un producto financiero. Para esto se realiza una llamada al cliente indicándole el producto con sus condiciones, y se le consulta al cliente si desea adquirirlo. Si el cliente acepta la propuesta se completa un formulario de registro del cliente, se valida la conformidad y se envía el expediente al área de finanzas; de lo contrario, el asesor agradece al cliente por su tiempo y termina la llamada.

Figura 5

Diagrama BPM para flujo promoción de un producto financiero



Arquitectura de Datos

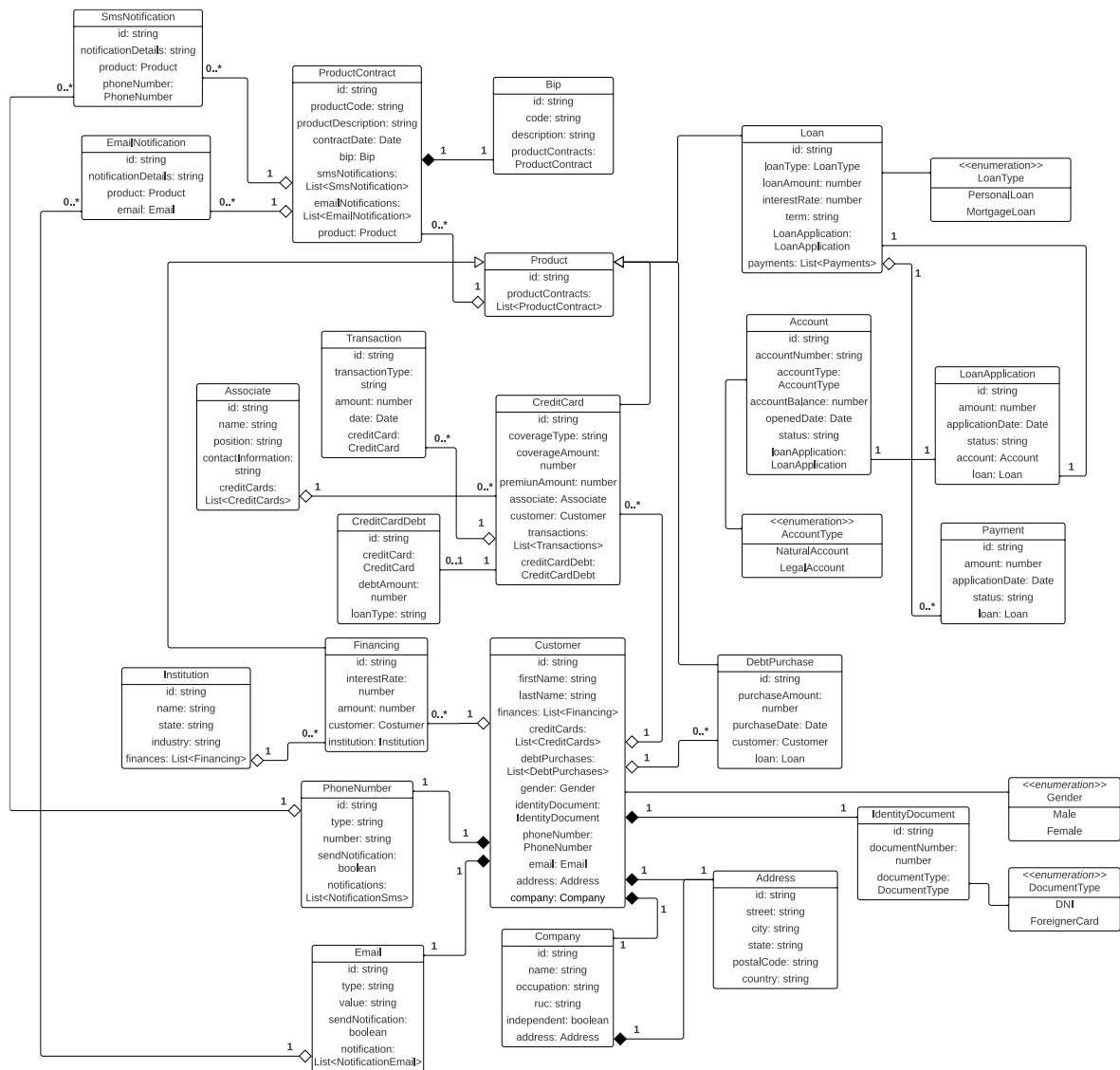
En la figura 6 se muestra las principales entidades que se definió en la arquitectura de la plataforma financiera estas entidades se utilizarán para cada uno de los productos o servicios.

- SMSNotification: Entidad que representa el registro de cada notificación enviada al cliente por mensaje de texto.
- EmailNotification: Entidad que representa el registro de cada notificación enviada al cliente por correo electrónico.
- ProductContract: Entidad que representa el contrato del producto que se crea al finalizar el proceso.
- BIP: Entidad que representa el código bip generado para el seguimiento del proceso por parte de la auditoria bancaria.
- Product: Entidad que representa un producto financiero.
- Loan: Entidad que representa el producto financiero ‘Préstamo personal’
- Account: Entidad que representa la cuenta donde se transfirió el monto del préstamo
- LoanApplication: Entidad que representa los detalles del préstamo ofrecido.
- Payment: Entidad que representa un pago de la deuda del préstamo.
- CreditCard: Entidad que representa el producto financiero ‘Tarjeta de crédito’
- Transaction: Entidad que representa una transacción realizada con la tarjeta de crédito.
- Associate: Entidad que representa al asesor financiero que aprobó la tarjeta de crédito
- CreditCardDebt: Entidad que representa la deuda que posee la tarjeta de crédito
- DebtPurchase: Entidad que representa el producto financiero ‘Compra de deuda’.

- Financing: Entidad que representa el producto financiero ‘Financiamiento’
- Institution: Entidad que presenta la institución externa que ofrece el financiamiento.
- Customer: Entidad que representa al cliente registrado en la institución financiera.
- PhoneNumber: Entidad que representa el número telefónico del cliente.
- Email: Entidad que representa el correo electrónico del cliente.
- Company: Entidad que representa la compañía donde trabaja el cliente.
- Address: Entidad que representa la dirección donde reside el cliente.
- IdentityDocument: Entidad que representa el documento de identidad del cliente.

Figura 6

Diagrama entidades del proyecto



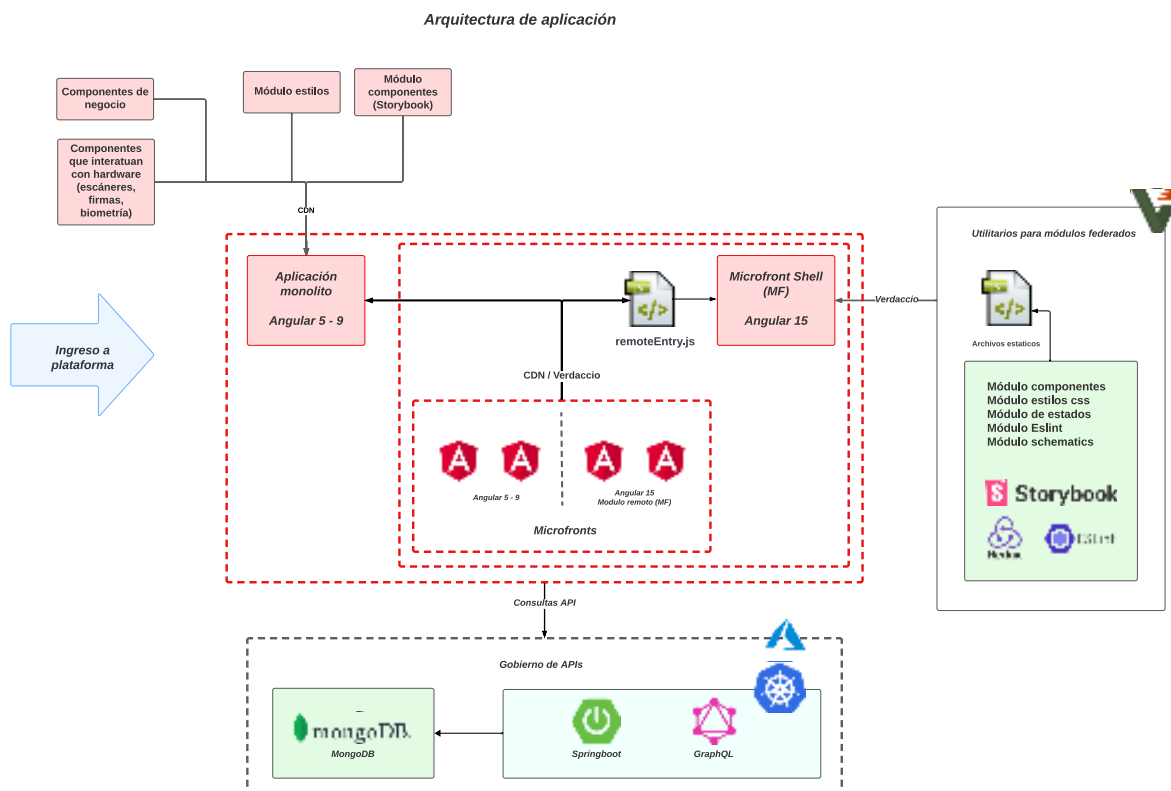
Arquitectura de Aplicaciones

La arquitectura de aplicaciones se diseñó bajo la premisa de mantener los proyectos de “Microfronts” antiguos (Angular 5-9) funcionando mientras se realice la migración de componentes al nuevo sistema. Estos seguirán siendo utilizados por el personal del banco y se les brindará soporte de incidencias en caso se requieran.

La figura 7 muestra la implementación de módulos federados, dentro de la aplicación monolito se creará un enrutamiento con el sufijo “/25hell” para agregar el Content Delivery Network (CDN) del “Microfront” 25hell de módulos federados, este proyecto es el que va a leer las subscripciones de los demás módulos federados, utilizando un archivo ‘remoteEntry.js’, que está alojado en la nube. Adicionalmente, para esta evolución tecnológica, se va a realizar proyectos Cross que van a proveer utilitarios que se van a utilizar para estandarizar el desarrollo en los “Microfronts” remotos, como la biblioteca de componentes, estilos, manejo de estados, herramientas de análisis de código y comandos para automatizar creación de componentes.

Figura 7

Diagrama de arquitectura de la aplicación



Arquitectura Tecnológica

La figura 8 muestra las herramientas a utilizar en el desarrollo de la solución, respecto al equipo “frontend”. Para el manejo del proyecto y comunicación de los equipos se utilizarán las

herramientas de la suite de Office 365, mientras que para el reporte y corrección de incidencias, documentación y CI/CD se utiliza la suite de Atlassian.

Con respecto al desarrollo, los “Microfrontends” utilizaran el framework Angular 15, además de Git para el control de versiones, Storybook para la creación y documentación de componentes y estilos a utilizar, Apollo Angular para el envío y recepción de solicitudes a GraphQL y Jest para realizar las pruebas unitarias.

En cuanto a los servicios en la nube, se utilizará Amazon Web Services (AWS) para el despliegue de los “Microfrontends” y Verdaccio con NPM para el registro de las bibliotecas.

Figura 8

Diagrama de arquitectura de tecnología

Arquitectura Tecnológica

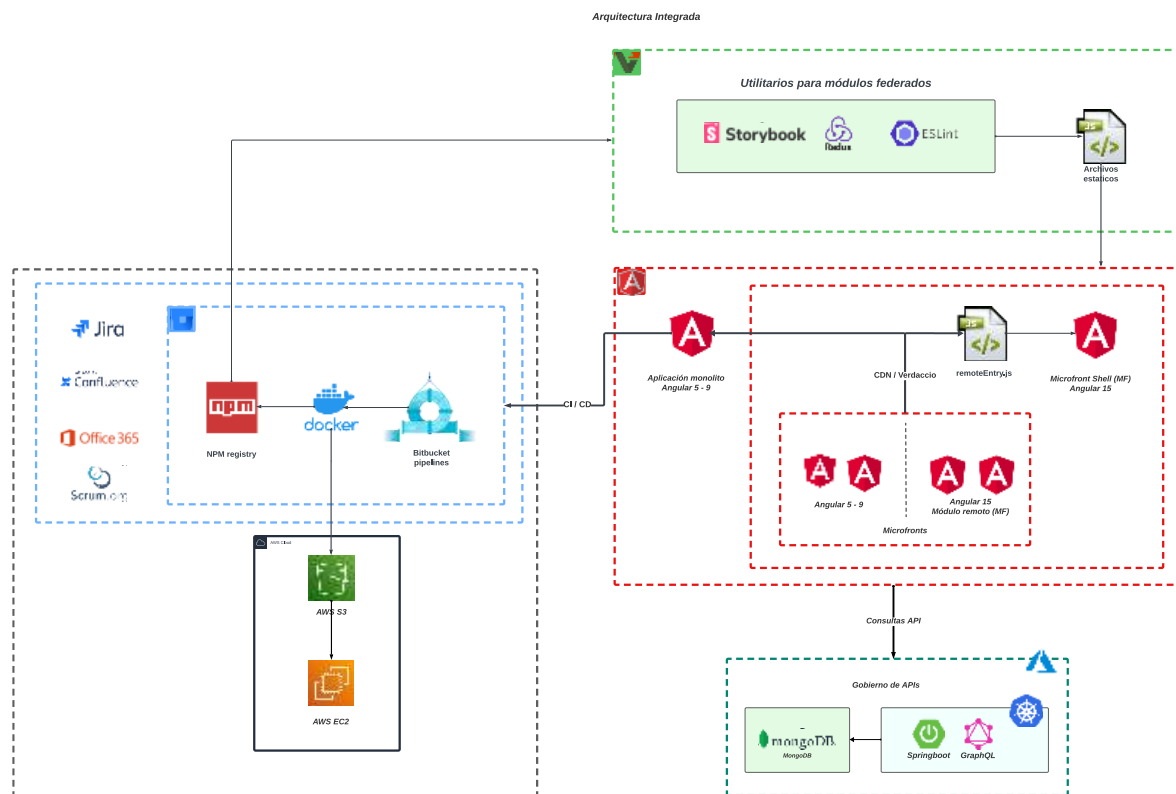


Arquitectura Integrada

Finalmente, la figura 9 representa la unión de las arquitecturas de negocio, datos, aplicaciones y tecnológica. Para lograr una mayor alineación, coherencia, eficiencia y seguridad en el sistema de información, así como facilitar su evolución y adaptación a los cambios del entorno, se han segmentado el diagrama en 4 secciones para especificar las tecnologías utilizadas por el equipo para la gestión del proyecto, desarrollo de los “Microfronts”, despliegue de paquetes utilitarios, y consumo de APIs.

Figura 9

Diagrama de arquitectura de tecnología



Desarrollo de la Solución

Una vez completado el proceso de diseño de la arquitectura, el gerente del proyecto dio inicio a la etapa de implementación. Como paso inicial, el arquitecto convocó una serie de reuniones con todo el equipo de desarrollo para informarles sobre la propuesta de arquitectura para esta evolución tecnológica. Se detalló el establecimiento de la versión 15 de Angular para los nuevos proyectos y la convivencia con los antiguos proyectos, la conformación de proyectos que son utilitarios para el nuevo ecosistema, el establecimiento del porcentaje de cobertura para pruebas unitarias en la aplicación cliente, y las bibliotecas a utilizar para la creación de nuevos proyectos y componentes. Estas charlas se realizaron en la primera semana del sprint, dependiendo de la disponibilidad de los colaboradores.

En esta etapa, el arquitecto desarrolló los proyectos utilitarios para la creación de nuevos "Microfronts", las versiones iniciales de las bibliotecas de componentes y estilos, así como el pipeline para la automatización de pruebas y despliegue. Estos últimos se agregaron en el script de creación de nuevos proyectos para automatizar estos procesos.

Una vez terminada la instrucción por parte del arquitecto, se procedió a establecer el nombre de los “Microfronts” a crear. En caso de ser un nuevo proyecto, se consultó al líder técnico del equipo para establecer la nomenclatura acorde a los servicios “REST” y “GraphQL” para mantener su uniformidad. Luego, se procedió con la creación de los repositorios asociados al proyecto para iniciar el desarrollo.

La aplicación monolito se implementó como la fuente de dependencias de las demás aplicaciones de forma de que sea la raíz de la jerarquía de componentes.

Por el lado de las bibliotecas de componentes y estilos construidos en “Storybook”. Se consultó al equipo de desarrollo por la existencia de componentes similares en otros proyectos, para abstraer los mismos a las nuevas bibliotecas. Esto permitió a reutilización de código funcional de otros proyectos para reducir tanto el tiempo de desarrollo como los errores en las pruebas.

Respecto a la aplicación Shell, se tomó en cuenta no renderizar ningún componente y solo realizar la identificación de los “Microfronts” que pertenecen a la federación de módulos, y que cuando se accede a uno de la federación, se active el prefijo “/site” en la jerarquía de rutas. Una vez seleccionado el “Microfront” de la federación de módulos, el Shell procede a cargar los componentes en un contenedor debajo de la barra superior, ya que es parte del dominio del monolito y sirve para realizar la redirección entre “Microfrontends” dentro y fuera de la federación, así como emitir valores al monolito padre.

Para la implementación de cambios, se realizaron revisiones de código en cada una de las solicitudes, lo cual ayudo a identificar malas prácticas, problemas de rendimiento, además de ayudar a mantener la coherencia y el cumplimiento de los estándares. Cuando se encontraron observaciones, el arquitecto colocó las observaciones e informó al desarrollador para que realice las correcciones necesarias para aprobar los cambios.

Una vez las funcionalidades fueron completadas, se realizaron pruebas unitarias para cada módulo, apuntando a mantener una cobertura de código mayor al 80%. Para facilitar esto, el arquitecto ofreció capacitaciones al equipo de desarrollo para conocer a mayor detalle cómo desarrollar pruebas unitarias de componentes, así como simular los servicios e integración con “Apollo-Angular”.

De este modo, el panorama desarrollado quedó conformado por los siguientes componentes:

- Aplicación monolito: Raíz de los módulos
- Aplicaciones utilitarias: Bibliotecas y Scripts

- Aplicación “Shell”: Configuración de los módulos
- Aplicaciones “Microfrontends”: Aplicaciones clientes

A continuación, se describen los “Wireframes” de los principales productos financieros.

En la figura 10 se puede visualizar el prototipo del servicio de compra de deuda. Donde se visualiza el registro que realiza el personal de la sucursal del banco para registrar la solicitud de un cliente. En este caso, los componentes y los estilos se recuperan por medio de las configuraciones del componente Shell, que a su vez se encarga de registrar todos los componentes que pertenecen a la federación de módulos.

Figura 10

Prototipo de producto compra de deuda

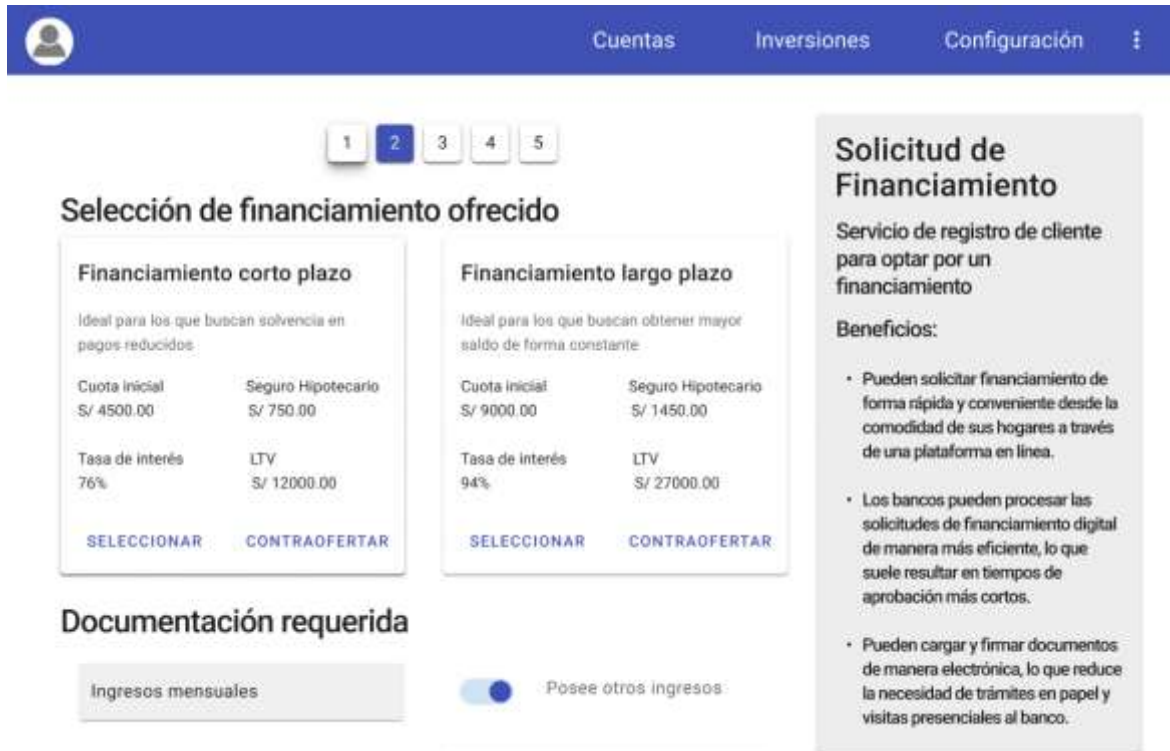
Al tener los “Microfronts” de la federación totalmente identificados, esta arquitectura permitió realizar la distribución y reutilización de métodos relacionados al negocio.

Esto se muestra en la figura 11, en donde el método de contraoferta está definido en solo un proyecto de las variantes de financiamiento, reduciendo la duración del desarrollo y facilitando el mantenimiento, ya que las actualizaciones deben realizarse en un solo lugar. Además, el uso del proyecto utilitario de manejos de estados a nivel de la federación permitió gestionar los estados de la federación de módulos de forma global, lo que fue útil al momento de realizar cambios de estado en las solicitudes de los clientes y mantener el estado coherente en toda la

aplicación. Esto evita la necesidad de pasar datos entre “Microfronts” de forma manual o realizar nuevamente consultas a los servicios para validar los cambios.

Figura 11

Prototipo de producto financiamiento



The image shows a mobile application prototype for a financing service. At the top is a blue navigation bar with a user profile icon, and links for 'Cuentas', 'Inversiones', and 'Configuración'. Below the navigation bar is a progress indicator with five steps, where step 2 is currently selected. The main content area is titled 'Selección de financiamiento ofrecido' and contains two side-by-side cards for 'Financiamiento corto plazo' and 'Financiamiento largo plazo'. Each card lists details like initial quota, mortgage insurance, interest rate, and LTV, with 'SELECCIONAR' and 'CONTRAOFERTAR' buttons. Below these cards is a 'Documentación requerida' section with a toggle for 'Ingresos mensuales' and a switch for 'Posee otros ingresos'. On the right side, there is a grey box titled 'Solicitud de Financiamiento' which describes the service and lists three benefits: rapid online application, efficient digital processing by banks, and the ability to upload and sign documents electronically.

Financiamiento corto plazo		Financiamiento largo plazo	
Cuota inicial	Seguro Hipotecario	Cuota inicial	Seguro Hipotecario
S/ 4500.00	S/ 750.00	S/ 9000.00	S/ 1450.00
Tasa de interés	LTV	Tasa de interés	LTV
76%	S/ 12000.00	94%	S/ 27000.00

Documentación requerida

☐ Ingresos mensuales ☒ Posee otros ingresos

Solicitud de Financiamiento

Servicio de registro de cliente para optar por un financiamiento

Beneficios:

- Pueden solicitar financiamiento de forma rápida y conveniente desde la comodidad de sus hogares a través de una plataforma en línea.
- Los bancos pueden procesar las solicitudes de financiamiento digital de manera más eficiente, lo que suele resultar en tiempos de aprobación más cortos.
- Pueden cargar y firmar documentos de manera electrónica, lo que reduce la necesidad de trámites en papel y visitas presenciales al banco.

Otro beneficio de la gestión de estados centralizada es en el aspecto de pruebas unitarias. Se simplifico la creación de pruebas para los reductores, las acciones y operaciones, lo que ayudo a reducir el tiempo tanto en la implementación y ejecución de las pruebas unitarias en los proyectos.

Esto se refleja en la figura 12, en donde la creación de las pruebas para las operaciones de calificación del cliente se pudo realizar de forma óptima utilizando Jest para simular consultas y respuesta de los servicios. Con respecto a pruebas de componentes, el tener un proyecto de biblioteca de componentes, se simplificaron las pruebas de las funcionalidades de los componentes en todo el dominio de la federación de módulos y el cumplimiento de los estándares definidos al inicio del proyecto en base a reglas definidas en “Eslint”.

Figura 12

Prototipo de producto tarjeta de crédito

Cuentas
Inversiones
Configuración

1
2
3
4

Información de tarjeta de crédito

CLIENTE

TARJETA

DOCUMENTACIÓN

TIPO DE MONEDA

Soles
Dólares

☒ Posee otra tarjeta de crédito

Fecha de vencimiento

Fecha de pago

Periodo de gracia

Envío de tarjeta física

☐ Recojo de tarjeta en sucursal
☒ Envío por entrega a domicilio

Solicitud de Tarjeta de crédito

Servicio de registro de cliente para la tarjeta de crédito

Beneficios:

- Permite a los clientes realizar compras y pagarlas en cuotas, lo que facilita la adquisición de bienes y servicios.
- Pueden elegir pagar el saldo total de la tarjeta o hacer pagos mínimos mensuales, lo que les da flexibilidad en su presupuesto.
- Acumulación de puntos, millas aéreas, dinero en efectivo u otros beneficios por cada dólar gastado con la tarjeta de crédito.

Por otro lado, si bien para esta evolución tecnológica se tuvo en cuenta no utilizar emisores para realizar el envío de datos, estos fueron necesarios para la transferencia de datos hacia el proyecto monolito en operaciones de redirección hacia fuera del dominio de la federación. Para establecer la comunicación de los componentes de la federación de módulos con el monolito, se crearon y utilizaron servicios en los proyectos utilitarios con las mismas características que se usan en los “Microfronts” que están fuera de la federación para emitir valores o estados.

Gestión del Proyecto

Alcance del proyecto

Se realizaron diversos artefactos que permiten establecer un resultado completo en función a los requerimientos solicitados. Entre ellos se tiene el EDT del proyecto, que define los paquetes de trabajo a realizar durante el proceso de desarrollo que sean un incremento para la plataforma, y a su vez desglosándose en tareas a realizar por el equipo en cada sprint.

Por otro lado, se establecieron criterios de aceptación dentro de la evolución tecnológica para tener poder visualizar la definición de hecho que debe tener el proyecto, esto facilita el trabajo a los equipos de desarrollo y pruebas al estar enfocados en el resultado final. Adicionalmente, se establecieron las exclusiones del proyecto para visualizar los puntos que no se van a contemplar en el desarrollo y evitar problemas respecto a las validaciones de funcionalidades.

Hoja de ruta del proyecto

Para que todo el equipo tenga presente el esquema de trabajo en forma global, se realizó una explicación detallada sobre el ciclo de vida del proyecto, desde su planificación, pasando por la fase de desarrollo, hasta su despliegue en la web.

Por otro lado, el equipo de gestión realizó un esquema de tareas a realizar para el primer trimestre del proyecto, indicando las tareas a desarrollar que no tengan dependencia con otras para evitar caer en bloqueantes. Posteriormente, se definió el enfoque a utilizar en el desarrollo del proyecto, indicando las fortalezas y debilidades de este.

Lista de Hitos

Adicionalmente, a las métricas definidas anteriormente, se realizó una lista de hitos que establecen los puntos de trabajo donde se van a realizar mediciones y verificar el avance del proyecto respecto al tiempo disponible. Esto sirve al equipo para brindar un estado del proyecto en distintos momentos y poder visualizar retrasos en el tiempo de entrega de las tareas.

Matriz de asignación de responsabilidades

Durante el desarrollo del proyecto, el equipo de gestión desarrolló una matriz de responsabilidades, donde se detalla la jerarquía de responsabilidades de cada miembro del equipo respecto a los paquetes de trabajo. Esto permite identificar a los miembros asignados e identificar el nivel de responsabilidad que tienen en cada tarea.

Gestión de calidad

Para establecer los estándares de calidad se consideró estándares internacionales reconocidos, como la ISO 9001, ISO 12207 e ISO 27001, debido a sus beneficios específicos en la gestión de calidad y el control en todas las etapas del software. A continuación, se detallarán los estándares adoptados para el desarrollo del proyecto.

ISO 9001

Establece un conjunto de requisitos que ayudan a las organizaciones a establecer procesos efectivos para la gestión de calidad en todas las etapas del desarrollo de software. Aplicado al desarrollo de la plataforma, provee un marco sólido para garantizar la coherencia en los procesos e identificación de impedimentos y problemas en el desarrollo de este.

ISO 12207

Se centra en los procesos del ciclo de vida, donde proporciona los lineamientos sobre como planificar, implementar, mantener y gestionar proyectos de desarrollo de software. Este punto

es de vital importancia para proyectos que involucren desarrollo de algún software debido a la gran cantidad de fases que estos poseen.

Adicionalmente, otra medida que se adoptó para garantizar una buena gestión de calidad fue el establecer los entregables y procesos sujetos a Revisión de Calidad utilizando la ISO 12207, que establece los procesos para el ciclo de vida del software. Esto permite segmentar el proyecto en entregables y establecer los puntos de validación adecuados en cada etapa de desarrollo. Esto permite que los entregables cumplan con los estándares de calidad definidos, ayudando a prevenir errores, reducir tiempo en las pruebas de calidad.

ISO 27001

La seguridad de la información es una preocupación principal al momento de desarrollar sistemas bancarios, por la información sensible que manejan estas aplicaciones. Por este motivo se adoptó la ISO 27001, que establece un marco de trabajo para la gestión de la seguridad de la información. Esto garantiza la identificación, evaluación y gestión de riesgos durante el desarrollo del proyecto.

Una vez se tengan identificados los perfiles que estarán en el desarrollo del proyecto, se desarrolló una Matriz de Roles y Responsabilidades de Calidad, esto con el propósito de identificar las responsabilidades de calidad relacionadas con cada rol definido para el proyecto.

Gestión de adquisiciones

Se definieron las fechas de adquisición de servicios a utilizar y otros aspectos importantes como métricas de rendimiento para tener registrado el costo en la facturación, roles de responsabilidad sobre los servicios a adquirir, suposiciones y restricciones, para tener más información sobre los aspectos que maneja el proveedor. Debido a que el proyecto es una evolución tecnológica y que el personal asignado al proyecto ya ha estado trabajando en proyectos anteriores de la entidad bancaria y poseen sus herramientas de trabajo, solo se han definido adquisiciones de servicios en la nube para el alojamiento y disponibilidad de recursos relacionados a los módulos federados.

Gestión de los recursos

Para conformar la gestión de recursos se estableció la identificación de recursos humanos y materiales. Por el lado de recursos materiales, no se necesita algún recurso puesto que los colaboradores poseen sus herramientas, así como de instalaciones de trabajo. Por otro lado, para el equipo de colaboradores para el desarrollo del proyecto, se realizó la identificación de roles que se necesitan para el proyecto, con el fin de realizar la propuesta formal al área de

gerencia y tener constancia sobre la cantidad de recursos a necesitar por rol. Una vez identificada la cantidad de recursos, se procedió a una planificación conjunta para establecer que personas van a conformar al equipo para el desarrollo del proyecto, y en el caso de que falte algún perfil, iniciar con la búsqueda de nuevos colaboradores que calcen con el perfil requerido.

Gestión de los riesgos

Respecto a la gestión de riesgos, se estableció la identificación de riesgos por cada fase del proyecto. Posteriormente, se realizó un documento de gestión de riesgos, que contiene los riesgos identificados, además de un monto monetario definido según el costo de la actividad, para tenerlo de reserva en caso presentar el determinado riesgo para accionar el protocolo de contingencia definido.

Otro punto definido en el documento es la frecuencia y tiempo, donde se identifica por cada riesgo, la frecuencia (en semanas) que se podría presentar el riesgo. Además de auditorías de los riesgos identificados para dar un seguimiento por parte de gerencia y tener un mejor panorama durante el desarrollo del proyecto.

Gestión de requerimientos

En el ámbito de requerimientos, la gerencia de proyecto envió un documento en donde se identifican los requerimientos necesarios en función de una investigación que se realizó por el arquitecto “Frontend” y el área de ciberseguridad, ordenando tanto los requerimientos funcionales como no funcionales. Posteriormente, se crearon las historias de usuario para cada requerimiento identificado y se estableció la forma de seguimiento para los requerimientos utilizando las ceremonias que ofrece el marco de trabajo Scrum, así como los reportes como la forma de validación para cada uno historia de usuario.

Gestión de alcance

Para garantizar una correcta definición del alcance del proyecto, se elaboró una estructura de desglose de trabajo (EDT), la cual permite segmentar el desarrollo por fases del proyecto y paquetes de trabajo. Cada paquete de trabajo define información como el objetivo, descripción del trabajo, roles asignados, fechas de inicio y fin, criterios de aceptación, supuestos y riesgos. Adicionalmente, se detalla la persona que debe dar la aprobación una vez que el paquete de trabajo este completo para finalizar con la entrega de este.

Gestión de costo

Respecto a la gestión de costo, se elaboró un documento de costos de procesos, en donde se detalla el tiempo en horas estimado de cada tarea del EDT por cada colaborador y/o servicio. Así como también el costo por hora y el costo total estimado por tarea. Adicionalmente, se contemplaron escenarios de costo excedido en umbrales de control, para establecer medidas de contingencia a nivel de gerencia.

Gestión del cronograma

Para la gestión del cronograma, se establecieron puntos importantes como el enfoque ágil guiado por Scrum, para la estimación y el desarrollo de las tareas. Además, se detallaron las principales herramientas de seguimiento del proyecto. A nivel gerencial se optó por utilizar la suite de Office 365 (MS Excel, MS Project), mientras que a nivel de desarrollo y pruebas se eligió la suite de Atlassian (Jira, Bitbucket, Confluence).

Posteriormente, se definieron los umbrales de variación en días para cada entregable definido en el EDT, esto es importante para el seguimiento del proyecto y establecer medidas de contingencia en el caso se presente un retraso que pueda afectar la fecha de entrega.

Gestión del proyecto

Con los datos recabados anteriormente, se desarrolló el documento a detalle del ciclo de vida del proyecto en MS Project, el cual identifica las revisiones de entregables por cada fase del proyecto. Aquí se detallan los criterios de entrada necesarios, así como los criterios de salida o artefactos generados, los recursos de colaboradores y servicios asignados, el tiempo (horas) estimado por cada tarea, y costo por colaborador y/o servicio. Siendo este documento el principal referente durante el proyecto. Adicionalmente, se elaboró un documento conteniendo el detalle los umbrales de variación de alcance, calendario y costos, con sus respectivas medidas de contingencia.

Resultado del Proyecto

Una vez terminado el desarrollo de cada “Microfront”, se realizó el despliegue de este utilizando el pipeline que se generó en la creación del proyecto con un script desarrollado por el arquitecto “Frontend”. Dentro de las principales tareas que ejecuta este pipeline están el análisis estático del código, la ejecución de pruebas unitarias y de componentes para validar si pasan el porcentaje de cobertura definido, y realizar el cambio de versiones de la aplicación monolito. Todo esto garantiza una transición suave y minimiza el riesgo de ocurrencia de errores en producción.

Por otro lado, para los cambios que se realizaron a nivel de proyectos utilitarios como bibliotecas de componentes y estilos, el arquitecto nombró a un colaborador para que recopile los cambios que se van a llevar a ambientes de producción, para posteriormente agregarlos a una sola rama y ejecutar la tarea de dentro del pipeline. Este proceso utiliza “Verdaccio”, un registro de dependencias de Node.js, para realizar la publicación y distribución de bibliotecas privadas de la plataforma.

Debido a la cantidad considerable de más 50 “Microfronts” coexistiendo en el ecosistema de la entidad financiera, sin contar los nuevos productos que se están siendo desarrollados con el nuevo enfoque, se decidió tomar el 20% del total desarrollado para la validación del resultado de esta sustentación. Esta segmentación permite una evaluación correcta del ecosistema de “Microfrontends” hasta donde se tiene desarrollado, sin comprometer la calidad del análisis y ni sobrecargar el proceso de revisión.

CONCLUSIONES Y RECOMENDACIONES

Conclusiones

La adopción de módulos federados dentro del ecosistema de “Microfrontends” proporciona un enfoque centralizado al momento de gestionar los componentes de la plataforma financiera. A nivel desarrollo, tal y como se detalló en el diagrama de la arquitectura de la aplicación en la figura 7, esta nueva arquitectura permite organizar las bibliotecas de componentes y estilos, pruebas unitarias y análisis de código estático, lo que permite versionar y mantener una calidad de código constante en todos los proyectos. El uso de gestión de estados dentro de la federación simplifico la gestión de datos entre los distintos “Microfronts” y componentes, lo que evitó realizar nuevamente consultas de servicio para obtener el valor actual de un determinado campo en etapas donde era necesario realizar una revisión de forma síncrona.

Otro aspecto de mejora fue en la automatización de procesos relacionados a la entrega y despliegue continuo. Esto debido al uso de utilitarios creados para la generación de nuevos “Microfronts”, donde solo se necesitaba establecer un nombre adecuado al tipo de producto que se iba a implementar para tener todo el proyecto totalmente configuración, incluyendo el pipeline de entrega continua, despliegue de los recursos estáticos en “Verdaccio” y su posterior actualización de versión que consume el proyecto Shell. Todos estos cambios permitieron reducir considerablemente el tiempo tanto de creación y configuración del pipeline desde el proyecto y el proveedor de servicios en la nube, así como en los cambios de versión que se debe realizar en el Shell.

Adicionalmente, esta implementación permitió establecer un alcance respecto al desarrollo de pruebas tanto al nivel de componentes como unitarias con la biblioteca “Jest”. Por un lado, el establecimiento de bibliotecas de componentes utilizando “Storybook” y “Stencil.js” facilitó centralizar la ejecución de pruebas en un solo módulo, para realizar el análisis de estos y realizar las correcciones de funcionalidades o formas de implementación en una sola base de código. Por otro lado, para las pruebas unitarias de los componentes creados en los propios “Microfronts” se mantuvo una cobertura del código del 80% manteniendo la funcionalidad necesaria para cumplir con legibilidad y coherencia en el código.

Recomendaciones

Para finalizar, de cara a continuar con la creación de nuevos “Microfronts” así como con la migración de “Microfronts” existentes hacia la federación de módulos, es recomendable priorizar el orden en función a aquellos que tengan las versiones más antiguas (Angular 5) y que presenten el mayor valor al negocio. Esto para evitar fallos en ambientes de producción que funcionen con bibliotecas deprecadas. Adicionalmente se debe tomar en cuenta los proyectos que tengan mayor dependencia de otros, y segmentarlos para realizar su migración al final de sus dependencias. Todo esto permite realizar las migraciones más críticas primero y visualizar los avances lo más pronto posible.

Otro aspecto por tomar en cuenta es realizar la documentación de los “Microfrontends” creados y migrados, para proporcionar una ayuda importante a futuros desarrollos de productos similares al guiarse y mantener las mejores prácticas, patrones de diseño y estándares de codificación permitiendo una coherencia en toda la federación de módulos. También es importante realizar la documentación de los cambios por cada incremento de las ramas que manejan los ambientes de desarrollo, pruebas y producción, para facilitar la identificación y evitar cambios innecesarios.

Una mejora por proponer es la inclusión de estándares de seguridad. Si bien cada “Microfront” de la federación tiene implementado un módulo de guardia para validar la autenticidad y expiración de los tokens de autenticación, es importante incluir protección contra vulnerabilidades conocidas y realizar la actualización regular de bibliotecas relacionadas. Esto es fundamental para proteger la información del usuario y cumplir con las regulaciones de privacidad de datos aplicables, así como establecer políticas de seguridad en los pipelines de integración y despliegue continuo.

A nivel de colaboradores, es importante mantener el enfoque de capacitaciones continuas dentro del equipo y proporcionar oportunidades de formación que les facilite conocer las tecnologías y las prácticas utilizadas en el desarrollo. Adicionalmente, se puede fomentar un sistema de charlas de conocimiento para afianzar la integración de nuevas tecnologías, arquitecturas y buenas prácticas. A medida que se introduzcan nuevos conceptos dentro del proyecto, es importante asegurarse que los nuevos miembros del equipo reciban el entrenamiento necesario para adoptar y utilizar estos conocimientos de manera efectiva. La inversión en formación y capacitación no solo mejora la competencia técnica de los colaboradores, sino que también fomenta un ambiente colaborativo y de crecimiento en el proyecto, lo que es esencial para tener éxito en un largo plazo.

REFERENCIAS

- Amazon Web Services. (2023). *¿Qué es la informática en la nube?*. AWS. Recuperado el 15 de septiembre de 2023, de https://aws.amazon.com/es/what-is-cloud-computing/?nc1=h_ls
- Asociación de Jóvenes Empresarios. (2009). *Productos financieros y alternativas de financiación*. Grupo Milán. https://www.camarazaragoza.com/docs/Guia_productos_financieros_y_alternativa_16602.pdf
- Banco Central de Reserva del Perú. (2022). *Glosario de términos económicos*. <https://www.bcrp.gob.pe/publicaciones/glosario.html>
- Beck K., Beedle M., Bennekum A., Cockburn A., Cunningham W., Fowler M., Grenning J., Highsmith J., Hunt A., Jeffries R., Kern J., Marick B., Martin R., Mellor S., Schwaber K., Sutherland J., & Thomas D. (2001). *Manifesto for Agile Software Development*. Recuperado el 03 de septiembre de 2023, de <https://agilemanifesto.org/iso/en/manifesto.html>

- Congreso de la República del Perú. (1996). *Ley N° 26702 de 1996. Por lo cual se expide Ley General del Sistema Financiero y del Sistema de Seguros y Orgánica de la Superintendencia de Banca y Seguros*
- Google. (2023, 15 de agosto). *What is Angular?*. Angular. Recuperado el 15 de septiembre de 2023, de <https://angular.io/guide/what-is-angular>
- Hu, B., & Zheng, L. (2016). Digital finance: Definition, models, risk, and regulation. *Development of China's Financial Supervision and Regulation*, 31–58. https://doi.org/10.1057/978-1-137-52225-2_2/COVER
- Jackson, C. (2019, 19 de junio). *Micro Frontends*. Martinfowler.com. Recuperado el 07 de septiembre de 2023, de <https://martinfowler.com/articles/micro-frontends.html>
- Jackson, Z. (2023). *Setting up Module Federation :: Module Federation Documentation*. Module Federation. Recuperado el 15 de septiembre de 2023 de <https://module-federation.io/docs/en/mf-docs/0.2/setup/>
- Object Management Group. (2023). *Business Process Model & Notation™ (BPMN™)*. OMG. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>
<https://www.omg.org/bpmn/>
- Peltonen, S., Mezzalana, L., & Taibi, D. (2021). Motivations, benefits, and issues for adopting Micro-Frontends: A Multivocal Literature Review. *Information and Software Technology*, 136, 106571. <https://doi.org/10.1016/J.INFSOF.2021.106571>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide The Definitive Guide to Scrum: The Rules of the Game*. Recuperado el 15 de septiembre de 2023, de <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>

ANEXOS

Plan de continuidad

Tabla 2

Tabla del plan de continuidad

Aspecto	Plan
Desarrollo	• Terminar con la migración del 80% restante de los proyectos
Mantenimiento y mejora continua	• Realizar el mantenimiento de los productos ya migrados. • Mantener la documentación actualizada

Identificación de riesgos	• Control de cambios para evitar revertir cambios en código. • Identificar posibles amenazas, como fallas en servicios externos • Implementar herramientas de trazabilidad de errores
Gestión del personal	• Establecer canales de comunicación en caso de interrupciones y dudas a nivel de servicios generales. • Capacitación a colaboradores

Plan de costos

Tabla 3

Tabla del plan de costos

Índice	Nombre de la tarea	Duración (días)	Cantidad de colaboradores	Costo/hora	Horas	Costo Total
1	Implementación de módulos federados basados en “Microfrontends”	331	20	30	2648	1588800
1.1	Planificación	32	10	30	256	76800
1.1.0	Análisis de la implementación actual de la plataforma	4	10	30	32	9600
1.1.1	Validación de gerencia de proyectos	0	4	30	0	0
1.1.2	Product Backlog	25	4	30	200	24000
1.2	Gestión de proyecto	25	10	30	200	60000
1.2.0	Antecedentes	4	6	30	32	5760
1.2.1	Marco Teórico	4	6	30	32	5760
1.2.2	Alcance del proyecto	4	6	30	32	5760
1.3.2	Gestión de costo	4	4	30	32	3840
1.3.3	Gestión de cronograma	4	4	30	32	3840
1.3.4	Gestión del proyecto	18	6	30	144	25920
1.3	Desarrollo	256	15	30	2048	921600

1.3.0	Desarrollo del proyecto Q1	88	15	30	704	316800
1.3.1	Desarrollo del proyecto Q2	88	15	30	704	316800
1.3.2	Desarrollo del proyecto Q3	74	15	30	592	266400
1.4	Despliegue	18	10	30	144	43200
1.4.0	Entrega documentación de "Microfrontends"	11	5	30	88	13200
1.4.1	Despliegue de "Microfrontends"	4	15	30	32	14400